Pricing & Promotion Decision Engine

定价与促销决策 Demo 完整执行计划

核心叙事：给定有效价格、促销状态、成本、库存和需求弹性，系统估计需求变化，模拟反事实，并生成需要实验验证的受约束 raise-and-test 定价候选。

项目定位	求职用 Decision Quant / Applied Scientist / Economist demo
主交付物	Streamlit web app + GitHub repo + methodology + executive memo
方法主线	需求估计 + 反事实模拟 + 利润优化 + A/B 验证设计
当前状态	数据清洗、EDA、需求估计、反事实优化、实验设计和 Streamlit MVP 已完成
技术栈	Python, pandas, statsmodels, scipy, Streamlit

版本：v1.1　日期：2026 年 4 月 21 日

目录

1 一句话结论 **2**

2 为什么不是纯预测，也不是选址 **demo** **2**

3 目标岗位与目标公司类型 **3**

4 业务问题定义 **4**

5 最小可行产品：**MVP** 架构 **4**

6 数据方案 **5**

7 **Relation to Prior Work** **11**

8 公式来源与解释边界 **12**

9 需求模型 **14**

10 反事实模拟器 **16**

11 优化器设计 **18**

12 不确定性与稳健性 **19**

13 **A/B test** 验证模块 **20**

14 **Web app** 设计 **20**

15 关键图表 **22**

16 项目文件结构 **22**

17 **README** 结构 **24**

18 执行路线图 **26**

19 一页 **Executive Memo** 模板 **27**

20 简历 **bullet** 与面试话术 **27**

21 关键风险清单 **28**

22 最终交付物清单 **28**

23 最终判断 **28**

参考文献 **29**

1 一句话结论

最终方向

不要做单纯销量预测 demo。做一个 **Pricing & Promotion Decision Engine**：它估计价格和促销对需求的影响，模拟有效价格与促销状态反事实，并在成本、库存、最低销量、最低毛利等业务约束下生成可审计的 raise-and-test 定价候选。

这比“我预测销量”强，因为它直接回答招聘方关心的问题：这个模型如何改变业务决策、候选动作的收益和风险是什么、上线前需要怎样验证？

这个项目不把“发明一个新的需求模型”作为卖点。它的定位是把 scanner-data demand estimation、pricing/revenue optimization、promotion response 和 experimentation 这些已有研究脉络，工程化成一个可交互、可复现、能产生业务建议的 app。

项目的最终目标不是展示模型复杂度，而是展示四种能力：

1. 会把业务问题转成可估计、可优化的决策问题；
2. 会处理自身价格弹性、交叉价格弹性、促销条件效应、价格和促销内生性、库存约束等真实问题；
3. 会把模型封装成业务方能使用的 web app；
4. 会把模型建议转成实验方案，而不是把观察性回归当成最终因果结论。

1.1 当前项目状态

截至 v1.1，项目已经从计划阶段推进到可运行 MVP。当前 repo 已包含：

- `01_data_cleaning.ipynb`、`02_eda.ipynb`、`03_demand_estimation.ipynb`、`04_counterfactual.ipynb` 和 `05_ab_testing_design.ipynb`；
- SKU-store-week panel、brand-size-store-week panel、demand modeling dataset、model coefficients、top recommendations 和 experiment candidates；
- Streamlit app，包括 Demand Model、Counterfactual Simulator、Profit Optimizer、Experiment Design、Limitations 和 Upload & Score 六个页面；
- methodology Markdown / TeX / PDF，以及 data cleaning、EDA、demand model、counterfactual 和 A/B validation 报告。

下一步不再是“开始建 app”，而是收口：先做 `07_cannibalization_robustness.ipynb` 检查同品牌跨包装规格互食，再补 tests、case study 和部署；若时间充足，再做 `08_causal_iv.ipynb`。

2 为什么不是纯预测，也不是选址 demo

2.1 Counterarguments

1. 纯预测回答的是“会卖多少”，不是“应该怎么做”。公司招聘定价、增长、市场设计、运营科学岗位时，核心关心的是价格、折扣、补贴、排序、库存分配等决策变量。

2. 选址是低频决策，反馈慢，实验难。电商和平台公司更常见的高频决策是价格、促销、推荐、广告预算和供需匹配。选址 demo 更像城市规划或零售地产分析，求职信号较窄。
3. 黑箱预测没有经济学优势。如果主线只是 XGBoost 预测销量，你与普通机器学习候选人没有明显差异。你的优势是因果识别、反事实推理、均衡直觉和约束优化。
4. 没有优化器的 **app** 只是 **dashboard**。招聘方需要看到“输入参数 - 反事实 - 推荐决策 - 风险验证”的闭环。

2.2 项目判别标准

项目类型	能回答的问题	求职信号
销量预测	下周销量是多少?	中等，偏 analyst/ML exercise
定价决策引擎	价格提高 5% 后，销量、收入、利润怎样变? 哪个价格值得测试?	高，直接对齐 decision quant
促销响应分析	打折是否真的增加利润? 需要怎样实验验证?	高，对齐 experimentation / causal inference
选址分析	新店应开在哪里?	小众，低频，难验证

3 目标岗位与目标公司类型

3.1 岗位定位

这个 demo 对齐以下岗位，而不是泛泛的数据分析岗位:

- Applied Scientist, Pricing / Marketplace / Optimization
- Economist, Causal Inference / Experimentation / Pricing
- Data Scientist, Pricing / Growth / Monetization
- Decision Scientist / Revenue Optimization Analyst
- Product Data Scientist, Marketplace / Promotion / Customer Growth

3.2 公司类型

公司类型	典型业务杠杆	Demo 对齐方式
电商/零售平台	价格、折扣、库存、推荐、促销预算	SKU-store-week 需求估计 + 利润优化
本地服务/餐饮 SaaS	商家定价、促销、用户增长、订阅留存	促销响应 + A/B test 设计
广告/营销科技	预算分配、投放 ROI、客户分群	uplift 建模 + constrained optimization
平台经济公司	补贴、匹配、动态定价、供需平衡	反事实模拟 + 约束优化
经济咨询	损害评估、反事实、政策模拟	识别问题 + 透明模型 + memo 写作

4 业务问题定义

4.1 核心问题

该 demo 要模拟一个真实定价团队面对的问题:

> 对于给定商品或品类，当前价格和促销策略是否合理？如果改变有效价格或促销状态，需求、收入、利润、库存风险会如何变化？在成本和库存约束下，哪些价格和促销状态值得作为 raise-and-test 候选？上线前需要怎样设计实验验证？

4.2 必须回答的业务问题

1. 当前价格是否过高或过低？
2. 促销提高的是销量、收入，还是利润？
3. 在给定 unit cost 下，候选价格是否仍有正 margin？
4. 在库存有限时，推荐策略是否造成 stockout？
5. 若价格上调或下调 5%，收入和利润分别如何变化？
6. 候选价格是否需要 A/B test 验证？实验需要多少样本？

5 最小可行产品：MVP 架构

5.1 六个核心模块

模块	输入	输出
1. 数据处理	交易数据、商品信息、日期、促销标记	SKU × store × week panel

模块	输入	输出
2. 需求估计	own price, competitor price, quantity, promo, SKU-store FE, week FE	自身价格弹性、交叉价格弹性、促销条件效应、预测误差
3. 反事实模拟	新有效价格、促销状态、成本、库存、需求冲击	预测销量、收入、利润、变化率
4. 优化器	目标函数、约束、价格范围、促销状态	raise-and-test 价格候选、促销状态、预期利润提升
5. 实验设计	baseline 指标、MDE、power、方差	样本量、实验周期、guardrail metrics
6. 应用层	app 选择器、场景输入、用户上传数据	交互式结果页、上传数据评分、风险提示

5.2 MVP 不做什么

- 不做复杂全栈前端。Streamlit 足够。
- 不把深度学习作为主模型。主模型必须透明。
- 不做“只有图表没有决策建议”的 dashboard。
- 不声称观察性价格弹性就是因果效应。
- 不伪造真实成本和库存；如果数据没有这些字段，就作为 scenario inputs。

6 数据方案

6.1 数据来源：Dominick’s Finer Foods, Cereals 品类

本项目选用 **Dominick’s Finer Foods (DFF)** 零售面板，范围锁定在 **Ready-to-Eat Cereals**（即食麦片）品类，覆盖全部门店与全部可用周次。数据由芝加哥大学 Booth 商学院 James M. Kilts Center 公开提供，时间范围约 1989–1997 年，包含约 85 家 Chicago 地区 Dominick’s 门店的周级 UPC-store 交易记录。

6.2 为什么选 Dominick’s 而不是其他公开零售面板

1. 字段完整度最高。DFF 同时含有 price、quantity (`MOVE`)、promo code (`SALE`)、profit margin (`PROFIT`)、week、store、UPC。利润函数所需的 unit cost 可由 price 与 PROFIT 反推，不必作为场景假设。这是所有公开零售面板中唯一同时满足这四类字段的数据集。
2. 学术与产业双重语境。DFF 是实证定价、产业组织、营销科学领域使用最广的公开面板。向 economist / applied scientist / pricing data scientist 岗位的面试官提到 “Dominick’s” 有即时识别度，

不需要额外解释数据来源。

3. 样本规模合适。约 85 家门店 × 约 400 周 × 单品类 UPC，足够估计价格弹性、品牌间交叉弹性和促销条件效应，同时可以在本地笔记本上完成处理，不依赖云端。
4. 促销字段精细。`SALE` 代码区分非促销、bonus buy、coupon、simple price reduction 等形式，允许把纯价格变动效应与条件 sale-code effect 分离——多数公开数据集只有二元 promo flag。

6.3 为什么选 Cereals 品类

维度	Cereals 品类的优势
学术锚点	Nevo (2001, *Econometrica*) "Measuring Market Power in the Ready-to-Eat Cereal Industry" 是实证定价方向的 landmark paper；本项目使用 DFF Cereals 作为同一品类和 scanner-data 语境下的透明决策工程化版本，不声称复刻 Nevo 的结构模型或原始数据
品牌结构	Kellogg's、General Mills、Post、Quaker、Ralston、Nabisco、private label 等 5–7 家势均力敌，交叉价格弹性有经济含义
利润叙事	品类 price-cost margin 约 40%–45%（Nevo 估计），是 market power 的典型案例，适合作为 memo 叙事支点
UPC 归一化	包装形式以盒装为主，size 维度少，`price per oz` 归一化简单
识别干扰	季节性弱、消费平稳、囤货 (stockpiling) 动机小，静态需求模型的跨期偏误较小
促销密度	促销频率高，能提供充足的价格内变动，有利于弹性识别

6.4 为什么排除其他候选品类

- **Soft Drinks**：2L 瓶 / 12 罐装 / 6 罐装 / 20oz 瓶等包装形式复杂，UPC 归一化工作量大；夏季季节性强，时间固定效应吸收不干净；消费者对促销的囤货行为 (Hendel-Nevo 效应) 会导致静态需求模型系统性低估真实弹性。
- **Beer**：税收、节假日与州法规冲击大，识别更复杂；部分时期促销折扣形式受法规限制。
- **Analgesics / Bath Soap**：品牌忠诚度极高、弹性小、价格内变动少，不足以做有信息量的弹性估计。
- **Frozen / Canned Foods**：促销频率低，价格变动不充分。
- 跨品类合并：不同品类弹性异质，合并估计单一 β 在经济上错误；按品类分别估计则实质是多个独立项目，不如做深一个品类。

6.5 数据基本信息

维度	取值
来源	UChicago Booth, James M. Kilts Center for Marketing — Dominick's Finer Foods Database
公开链接	https://www.chicagobooth.edu/research/kilts/research-data/dominicks
品类	Ready-to-Eat Cereals（交易文件前缀 `wcer`，UPC 映射 `upccer`）
时间范围	约 1989–1997 年，周级
门店数	约 85 家 Chicago 地区门店
UPC 数量	UPC 映射表 490 行；Step 2 清洗后交易面板保留 486 个有有效交易记录的 UPC，可继续聚合到 brand × size 主 SKU
主要品牌	Kellogg's, General Mills, Post, Quaker, Ralston, Nabisco, private label
核心字段	`UPC`，`STORE`，`WEEK`，`MOVE`，`PRICE`，`PROFIT`，`SALE`，`QTY`
可反推字段	`unit_price` = `PRICE`/`QTY`；`unit_cost` = `unit_price` × (1 - `PROFIT`/100)
规模估计	Step 2 清洗后为 4,654,156 行，覆盖 486 UPC、93 stores、366 weeks、36,199 UPC-store pairs；每对中位覆盖 78 周

6.6 字段结构与 DFF 对应关系

项目主面板的最小字段及其与 DFF 原始字段的映射如下：

项目字段	DFF 对应	含义	备注
product_id	UPC	商品 ID	可聚合到 brand
store_id	STORE	门店 ID	必须，用于 SKU-store FE
week	WEEK	周序号	DFF 自定义周计数
price	PRICE/QTY	有效单位销售价格（美元）	价格弹性主变量
price_per_oz	price/size_oz	单位容量价格	用于跨品牌比较
bundle_price	PRICE	捆绑销售价格（美元）	用于审计原始价格
quantity	MOVE	周销售单位数	单位数而非重量
promo	SALE	促销代码	MVP 转为二元 indicator
cost	unit_price × (1 - PROFIT/100)	单位成本	由 DFF gross margin 反推
brand	由 UPC 映射	品牌	用于异质性与交叉弹性
size_oz	由 UPC 映射	包装容量（盎司）	归一化 price per oz 用
category	固定为 cer	品类	本项目锁定 Cereals
inventory	数据缺	库存	作为 scenario input
competitor price	同 store-week 内其他品牌均价	竞争价格	用基准份额加权

6.7 数据质量底线

- 不使用只有销量而缺失价格的周；
- 不使用只有浏览 / 曝光而无交易结果的数据；
- 不使用纯合成数据；DFF 为真实零售面板；
- PROFIT 字段缺失或异常的记录不进入利润计算；

- 库存字段 DFF 不提供，作为 scenario input 在 app 中明确标注；
- 所有聚合与过滤决策（UPC 过滤阈值、最小周数、异常价格规则）写入 `rawData/README.md` 或 `reports/data_cleaning_summary.md`，并在根目录 `README.md` 中链接，保证可复现。

6.8 Step 2 数据诊断记录

第一轮 notebook 清洗已经产出 `data/processed/sku_store_week_panel.parquet`、`reports/data_cleaning_summary.md` 和 `reports/figures/data_quality_wcer.png`。进入模块化前，以下数据事实需要固定到文档和清洗规则中：

- `PRICE=0` 与 `MOVE=0` 几乎完全重合：raw 中所有 1,850,703 个零销量行都同时为零价格；另有 677 行 `PRICE=0`，`MOVE>0`，视为无效价格记录。因此主需求样本继续使用 `PRICE>0` 和 `MOVE>0` 过滤。
- `SALE` 除 B/S/C 外还出现 `G` 和 `L`。其中 `G` 频次足够高，MVP 二元 `promo` 中计为促销，并在分类分析中保留为 `unknown_promo`；`L` 只有 1 行，并入缺失。
- `PROFIT` 中位数为 13.7%，validation sanity range 应使用 $[5, 30]$，而不是 $[20, 50]$；负利润或大于等于 99% 的记录不进入成本和利润计算。
- `COM_CODE` 对所有 Cereals UPC 都是 311，不存在需要额外处理的子品类混杂。
- `SIZE` 严格 OZ 正则可解析率为 72.2%。`11.25O`、`1.25 O`、`19.25Z` 这类截断单位要用鲁棒正则处理；`ASST`、`1 CT`、`end` 等脏值要从 `price_per_oz` 和竞争价格构造中剔除或人工修复。
- 清洗后样本覆盖 486 UPC、93 stores、366 weeks 和 36,199 UPC-store pairs；每对中位覆盖 78 周，足以支持 SKU-store pair fixed effects。

6.9 当前已完成的实证产出

当前项目已经完成 Step 2–Step 5 的主要流水线，关键结果需要同步到 README、methodology、app 和报告中：

- **EDA 与聚合**：`brand_size_store_week_panel.parquet` 已生成，brand-size 粒度作为 baseline，UPC 粒度作为 robustness；brand mapping 经 DESCRIP 规则与 manufacturer code 交叉验证。
- 需求估计：baseline 估计得到 $\widehat{\beta}_{own} \approx -1.75$, within R^2 约 0.75；这里的 within R^2 指 fixed-effects panel regression fit，不是 out-of-sample predictive accuracy。加入竞争价格后 $\widehat{\beta}_{own} \approx -1.73$、$\widehat{\beta}_{cross} \approx 0.65$、促销 sale-code 系数约 0.43 log points，对应 $\exp(0.43) - 1 \approx 54\%$ 的条件销量差异。UPC robustness 给出 $\widehat{\beta}_{own} \approx -1.90$、$\widehat{\beta}_{cross} \approx 0.50$。
- 预测与重转换：log-demand 的 level prediction 使用 Duan smearing；cell-anchored counterfactual 从 observed level baseline 出发，因此不再额外乘 smearing factor，避免重复校正。
- 反事实优化：eligible cells 中 98.5% 的候选动作贴到上界 price guardrail。该结果必须解释为模型诊断和 raise-and-test candidates，而不是 production deployment recommendations。

- 实验设计：top candidates 多数属于 high-risk / underpowered single-store flights，说明优化器候选动作必须通过更长周期或更大范围实验验证。

6.10 处理流程

1. 从 Kilts Center 下载 `wcer.csv`（交易）与 `upccer.csv`（UPC 映射）；
2. 清洗异常价格、零销量、重复 (UPC, store, week) 记录；主需求模型仅使用 $Q > 0$ 的周；
3. 构造有效单位价格 `unit_price` = `PRICE`/`QTY`，并用它作为需求模型中的 price；
4. 构造 `unit_cost` = `unit_price` × (1 - `PROFIT`/100)；
5. 由 UPC 映射表补充 brand 与 size_oz，归一化 `price_per_oz`；
6. 按 brand × size_oz 聚合（或直接 brand 级），降低 UPC 噪声；但聚合可能把同一 brand-size 内不同 UPC 的价格、促销和 bundle 组合变化混进需求反应，因此 UPC 粒度结果必须作为 robustness check；
7. 构造 SKU-store-week panel，计算 log price、log quantity、二元 promo indicator；
8. MVP 不单独模拟连续 discount depth；若增强版加入折扣深度，必须先显式估计对应系数；
9. 构造同一 store-week 内其他品牌的加权 `price_per_oz` 作为竞争价格控制；权重使用 pre-period 或滚动滞后销量份额，不使用当期销量份额；
10. 过滤样本过少的 SKU（如少于 20 周观察）；
11. 按时间划分 train / holdout（如最后 20% 周留作 holdout）；
12. 保存 processed panel 为 parquet，供模型与 app 调用。

6.11 Brand-size 聚合公式

当 UPC 粒度观测聚合到 brand × size_oz × store × week 时，令 g 表示 brand-size cell，$u \in g$ 表示属于该 cell 的 UPC。聚合规则必须固定，避免后续模型变量来源不清：

$$Q_{gst} = \sum_{u \in g} MOVE_{ust}. \tag{1}$$

销售额先在 UPC 层根据 DFF bundle price 与 bundle quantity 计算，再聚合：

$$Sales_{gst} = \sum_{u \in g} \frac{PRICE_{ust} \times MOVE_{ust}}{QTY_{ust}}. \tag{2}$$

聚合后的有效价格是 revenue-per-unit，也等价于按销量加权的有效单位价格：

$$P_{gst}^{eff} = \frac{Sales_{gst}}{Q_{gst}}. \tag{3}$$

聚合后的 unit-cost proxy 按销量加权：

$$c_{gst} = \frac{\sum_{u \in g} c_{ust} MOVE_{ust}}{\sum_{u \in g} MOVE_{ust}}. \tag{4}$$

baseline 促销变量使用 any-promo indicator：

$$Promo_{gst}^{any} = \mathbf{1}\left\{\sum_{u \in g} Promo_{ust} > 0\right\}, \tag{5}$$

同时保留按销量加权的促销占比作为诊断变量：

$$Promo_{gst}^{share} = \frac{\sum_{u \in g} Promo_{ust} MOVE_{ust}}{\sum_{u \in g} MOVE_{ust}}. \tag{6}$$

6.12 竞争价格构造

竞争价格用同一门店、同一周内其他品牌的单位容量价格构造：

$$P_{ist}^{comp} = \sum_{b \neq B(i)} w_{bs}^{base} P_{bst}^{oz}. \tag{7}$$

其中 $B(i)$ 是商品 i 所属品牌，P_{bst}^{oz} 是品牌 b 在门店 s、周 t 的加权平均 `price_per_oz`。权重 w_{bs}^{base} 使用 pre-period 或滚动滞后销量份额，并在当前周 t 之前确定；不使用当期销量份额，避免把当期需求冲击机械地放进竞争价格变量。若某门店历史销量不足，MVP 可退回到其他品牌等权平均。

7 Relation to Prior Work

这个项目的研究问题不是全新的。价格弹性、促销反应、品牌替代和利润优化都已经有成熟文献和商业系统。项目的价值在于：用公开 DFF 数据做一个透明、可运行、可解释的 decision engine，把研究问题从“估计需求”推进到“生成候选价格、解释风险、设计实验”。

7.1 Scanner Data 与零售定价研究

Dominick's Finer Foods 数据本身来自 UChicago Booth Kilts Center 的 store-level shelf management and pricing research 合作，包含多品类、门店级、周级 UPC movement data。它适合作为 MVP 数据源，不是因为它最新，而是因为字段足够接近真实定价团队需要的最小闭环：价格、销量、促销代码、毛利、UPC、门店和周次。

因此，本项目不是从零构造一个 toy dataset，而是站在 scanner-data retail pricing 的经典经验研究语境上。README 中应明确写出：DFF 是历史数据，适合方法演示和 portfolio 复现；上线到真实业务时，需要替换为当前业务的 transaction / inventory / campaign / cost 数据。

7.2 Differentiated Demand 与 Cereal 文献

BLP (Berry, Levinsohn, and Pakes, 1995) 和 Nevo (2001) 是差异化产品需求估计的核心参照。它们关心的是结构需求、产品替代、市场势力和价格-成本边际，而不是简单预测下周销量。Cereals 品类有

多个大品牌、明显产品差异和可解释的交叉替代关系，所以适合把 own-price elasticity 与 cross-price elasticity 作为模型输出。

本项目有意识地不做完整 BLP 或品牌对品牌的结构替代矩阵。MVP 使用 SKU-store FE、week FE、聚合竞争价格和二元促销项，目标是做透明决策支持，而不是估计一个可发表的结构产业组织模型。这个边界需要写清楚，反而会让项目显得更可信。

7.3 Promotion、库存行为与静态模型边界

促销不只是价格下降。Hendel and Nevo (2006) 强调，temporary sales 可能引发消费者 stockpiling；如果忽略这种动态行为，静态需求模型可能错误衡量长期价格反应。DFF 的 `SALE` 字段让本项目可以把促销状态显式纳入模型，但 MVP 仍然是静态周级模型。

因此，文档和 app 必须保留两层 caveat：第一，促销状态本身可能内生；第二，促销周销量提升可能包含跨期购买转移。MVP 的推荐适合做短期决策模拟，不能直接解释为长期均衡销量提升。

7.4 Revenue Optimization 与商业决策系统

Revenue management / pricing optimization 文献把需求曲线、边际成本、容量或库存约束放进同一个优化问题。Talluri and van Ryzin (2004) 与 Phillips (2005) 是这条线的经典参考。本项目沿用这个工程逻辑：先估计 price-response，再模拟 counterfactual，再在 margin、inventory、price band 和 promo cost 约束下优化。

与商业定价软件相比，本项目的重点不是覆盖所有企业级功能，而是把黑箱系统拆成一个可读的求职 demo：数据清洗、弹性估计、利润函数、优化器、不确定性、识别 caveat 和 A/B test 验证全部在一个 repo 里闭环。

7.5 本项目的可讲创新点

定位句

This is not a new econometric estimator. It is an applied decision product that translates classic scanner-data demand estimation and pricing optimization into a transparent, interactive, experiment-aware tool.

面试或 README 中可以把差异化讲成四点：

1. 从研究到决策。不是只报告弹性，而是把弹性放进利润函数和约束优化器；
2. 从单品价格到品类竞争。不是只估 own-price response，而是加入 cereal 品牌间的聚合 cross-price term；
3. 从观察性估计到实验验证。不把 FE 回归包装成最终因果结论，而是输出 A/B test 或 rollout 设计；
4. 从 **notebook** 到 **app**。把模型变成业务方可以调 price、promo、cost、inventory 和 competitor price 的交互工具。

8 公式来源与解释边界

8.1 三类公式

本项目中的公式分成三类，必须在 methodology 和面试叙事中分清：

1. **DFF accounting transformations**：有效价格、销售额和 unit-cost proxy 来自 DFF 字段与 manual 的会计定义；
2. 标准计量与定价公式：log-log fixed-effects demand、Duan smearing、A/B test 样本量、constrained pricing optimization 都来自成熟方法；
3. 项目实现选择：竞争价格指数、cell-anchored counterfactual、grid search 和 scenario overlay 是本项目为决策支持做的工程化实现，不是新的理论模型。

8.2 DFF 字段到模型变量

DFF 有时记录 bundle price，因此有效单位价格定义为：

$$P_{ist}^{eff} = \frac{PRICE_{ist}}{QTY_{ist}}. \tag{8}$$

对应销售额为：

$$Sales_{ist} = \frac{PRICE_{ist} \times MOVE_{ist}}{QTY_{ist}}. \tag{9}$$

unit cost 不是观测到的边际成本，而是由 DFF gross-margin 字段反推的 average-acquisition-cost proxy：

$$c_{ist} = P_{ist}^{eff}\left(1 - \frac{PROFIT_{ist}}{100}\right). \tag{10}$$

因此，利润模拟是方向性 decision simulation，不应被解释为精确会计利润或 observed marginal-cost optimization。

8.3 需求、重转换与反事实

主需求方程是 reduced-form constant-elasticity panel approximation，不是 BLP / Nevo 结构估计，也不是品牌对品牌的完整替代矩阵。BLP 和 Nevo 的作用是提供 differentiated-products demand 与 cereal market 的研究语境，不是本项目方程的直接来源。

Duan smearing correction

$$\widehat{S} = \frac{1}{N}\sum_{n=1}^{N}\exp(\widehat{\epsilon}_n) \tag{11}$$

只用于把 fitted log-demand 转回 level prediction。app 优化器中的 anchored counterfactual 从 observed level baseline 出发：

$$\widehat{Q}(p', m', p^{comp\prime}) = \bar{Q}_{cell}\ \exp\left[\widehat{\beta}_{own}\Delta\log p + \widehat{\beta}_{cross}\Delta\log p^{comp} + \widehat{\theta}\Delta m\right], \tag{12}$$

因此不再额外乘 smearing factor。默认情况下竞争价格固定，$\Delta \log p^{comp} = 0$；只有当用户显式运行 competitor-price scenario 时，交叉价格项才进入反事实变化。

8.4 收入、利润与 guardrail 解释

inventory cap 必须同时约束 revenue 和 profit。令

$$Q^{sold}(p', m', p^{comp\prime}) = \min\{\widehat{Q}(p', m', p^{comp\prime}), Inventory\}, \tag{13}$$

则

$$Revenue(p', m', p^{comp\prime}) = p'\, Q^{sold}(p', m', p^{comp\prime}), \tag{14}$$

$$Profit(p', m', p^{comp\prime}) = (p' - c)\, Q^{sold}(p', m', p^{comp\prime}) - Fm'. \tag{15}$$

当优化器频繁绑定上界 price guardrail 时，输出应解释为 **raise-and-test candidate generated by a constrained local demand model**，不是 automatic deployment recommendation。guardrail binding 表明在估计的 constant-elasticity 需求曲线和 DFF cost proxy 下，无约束利润目标偏好比历史支持范围更高的价格。

9 需求模型

9.1 透明基准模型

主模型采用 SKU-store 固定效应和周固定效应的对数需求模型。MVP 中，价格变量是有效单位成交价，促销变量是二元 indicator；连续折扣深度不在同一模型里混用。由于 $\log(Q)$ 要求正销量，主需求模型过滤 $Q > 0$ 后估计的是 **observed positive-sales cells** 上的条件需求，而不是所有可能 UPC-store-week 的无条件品类需求。零销量和缺失 movement week 可在 robustness 中用补全面板、$\log(Q+1)$ 或 availability indicator 处理。

$$\log(Q_{ist}) = \alpha_{is} + \gamma_t + \beta_{own}\log(P^{eff}_{ist}) + \beta_{cross}\log(P^{comp}_{ist}) + \theta Promo_{ist} + X'_{ist}\delta + \epsilon_{ist}. \tag{16}$$

其中：

- Q_{ist} 是商品 i 在门店 s、周 t 的销量；
- P^{eff}_{ist} 是有效单位销售价格；
- P^{comp}_{ist} 是同一 store-week 中竞争品牌的加权平均 `price_per_oz`；
- α_{is} 是 SKU-store pair 固定效应，吸收稳定的商品-门店差异；
- γ_t 是周固定效应，吸收全市场季节性与共同冲击；
- $Promo_{ist}$ 是二元促销状态；

- X_{ist} 是可选控制项，MVP 可设为空；增强版可加入节假日、品牌 × 月份季节项、store-specific trend 或滞后需求控制；
- β_{own} 是自身价格弹性；
- β_{cross} 是交叉价格弹性；
- θ 是条件 sale-code effect，以 log points 计量；对应百分比效应是 $\exp(\theta) - 1$，不是 θ 本身。

为什么使用 SKU-store FE

Dominick's 是 UPC × store × week 面板。不同门店的商圈收入、客流、货架陈列和基准价格水平可能长期不同；只用 SKU FE 会把这些门店间价格差异误记为需求弹性。α_{is} 比单独加入 SKU FE 和 store FE 更保守，因为它允许每个商品在每家门店有自己的基准需求水平。

9.2 输出解释

弹性解释

如果 $\widehat{\beta}_{own} = -1.8$，则自身价格上升 1% 与销量下降约 1.8% 相关，条件是控制了 SKU-store 固定效应、周固定效应、竞争价格和促销状态。若 $\widehat{\beta}_{cross} > 0$，则竞争品牌价格上升与本商品销量上升相关，符合替代品直觉。若 $\widehat{\theta} = 0.43$，它表示促销代码对应约 $e^{0.43} - 1 \approx 54\%$ 的条件性销量差异，而不是 43% 的直接 uplift。这些估计可以支持决策模拟，但不能自动解释为最终因果效应。

9.3 重转换修正

由于模型估计的是 $\log(Q)$，不能直接把线性预测值取 $\exp(\cdot)$ 当作无偏销量预测。MVP 使用 Duan smearing correction：

$$\widehat{S} = \frac{1}{N} \sum_{n=1}^{N} \exp(\widehat{\epsilon}_n). \tag{17}$$

给定线性预测值 $\widehat{\eta}$，销量预测为：

$$\widehat{Q} = \widehat{S} \cdot \exp(\widehat{\eta}). \tag{18}$$

增强版可以按 promo 状态、价格分位、品类、品牌或 SKU 分组计算 smearing factor，例如分别估计 $\widehat{S}_{m=0}$ 和 $\widehat{S}_{m=1}$，以处理促销周误差方差更大的情况。第一版先用全样本修正，避免系统性低估销量、收入和利润。

9.4 识别 caveat

价格和促销通常都不是随机设定。高需求时期公司可能涨价，低需求时期可能促销；促销也可能来自库存压力、厂家补贴、category captain 排期、竞争对手行动或门店层面的需求预测。这会导致价格和促销系数都存在内生性。因此，文档和 app 必须写清楚：

Observed price and promotion variation are not automatically causal. Own-price elasticity, cross-

price elasticity, and conditional sale-code effects are used for decision support and should be validated through randomized price/promotion tests, staggered rollouts, credible instruments, or difference-in-differences designs before full rollout.

9.5 识别策略升级

MVP 先使用 SKU-store FE、week FE、竞争价格和促销控制，重点交付透明的决策闭环。增强版把识别问题单独作为模块，而不是把观察性回归包装成最终因果结论。

策略	实现方式与作用
价格残差诊断	回归 $\log(P_{ist}^{eff})$ on promo、SKU-store FE、week FE，报告 first-stage-like F statistic 和 R^2，说明价格变动有多少来自促销与固定效应之外
Hausman IV	用同一 UPC 或品牌在其他门店同周的价格作为 focal store 价格工具变量；适合需求冲击主要是 store-specific 的设定，但需要警惕全城促销或共同需求冲击
成本冲击 IV	使用滞后 unit cost 或外部谷物 / 包装材料 PPI 作为工具变量；不直接使用 contemporaneous unit cost，因为它由 price 和 margin 机械反推，可能与价格共同决定
实验验证	对候选价格或促销状态做小流量随机实验、geo rollout 或 staggered rollout，把观察性建议转化为可验证决策

10 反事实模拟器

10.1 预测需求

给定新的有效单位价格 p'、促销状态 $m' \in \{0,1\}$ 和竞争价格 p^{comp}，模型层面的 level 预测为：

$$\widehat{Q}(p', m', p^{comp}) = \widehat{S} \cdot \exp\left(\widehat{\alpha}_{is} + \widehat{\gamma}_t + \widehat{\beta}_{own}\log(p') + \widehat{\beta}_{cross}\log(p^{comp}) + \widehat{\theta}m' + X'_{ist}\widehat{\delta}\right). \tag{19}$$

实际 app 优化器使用 cell-anchored relative response，先从该 cell 的历史平均销量出发，只应用候选价格、竞争价格和促销状态相对于 baseline 的变化：

$$\widehat{Q}_{anchor}(p', m', p^{comp\prime}) = \bar{Q}_{cell}\exp\left[\widehat{\beta}_{own}\Delta\log p + \widehat{\beta}_{cross}\Delta\log p^{comp} + \widehat{\theta}\Delta m\right]. \tag{20}$$

默认 MVP 中竞争价格保持不变，即 $\Delta\log p^{comp} = 0$；只有当用户显式运行 competitor-price scenario 时，交叉价格项才进入反事实变化。因为 $\bar{Q}_{cell}$ 已经是 level quantity，anchored 反事实不再额外乘 smearing factor，避免把 level correction 重复计算。

MVP 不把连续折扣 d 同时放进 $\log(p'(1-d))$ 和促销项，避免重复计算折扣效应。若增强版需要 discount slider，必须先构造并估计 `discount_depth` 的连续系数，再用对应模型做反事实。竞争价格 p^{comp} 默认取当前 store-week 的竞争品牌加权 `price_per_oz`；高级情景可以让用户模拟竞争对手价格变化。

10.2 收入和利润

$$Q^{sold}(p', m', p^{comp}) = \min\{\widehat{Q}_{anchor}(p', m', p^{comp}), Inventory\}, \tag{21}$$

$$Revenue(p', m', p^{comp}) = p' \cdot Q^{sold}(p', m', p^{comp}), \tag{22}$$

$$Profit(p', m', p^{comp}) = [p' - c] \cdot Q^{sold}(p', m', p^{comp}) - F \cdot m'. \tag{23}$$

其中 c 是单位成本。DFF 中 c 由有效单位价格和 gross margin 反推，是 average-acquisition-cost proxy，不是 observed marginal cost。库存字段 DFF 不提供，因此 $Inventory$ 作为 scenario input。F 是可选促销固定成本，MVP 默认 $F = 0$，增强版可由用户输入陈列、人工或 coupon 运营成本。注意 revenue 和 profit 都必须使用同一个 capped sold quantity，不能 revenue 用 predicted demand、profit 才 cap inventory。

当 app 展示候选动作的 absolute profit 时，促销固定成本直接进入候选利润：

$$Profit(p', m') = (p' - c)Q^{sold}(p', m') - Fm'. \tag{24}$$

当 app 展示相对于观察到的 baseline (p_0, m_0) 的 profit lift 时，促销固定成本必须差分：

$$\Delta Profit = \left[(p' - c)Q^{sold}(p', m') - Fm'\right] - \left[(p_0 - c)Q^{sold}(p_0, m_0) - Fm_0\right]. \tag{25}$$

因此，F 只有在候选动作和 baseline 的促销状态不同的时候才改变 incremental lift；如果二者都促销或都不促销，固定成本在差分中抵消。

10.3 Scenario overlay 与 shock 定义域

Streamlit app 在冻结的需求模型之上提供 business shock overlay，而不是重新估计模型。记 δ_q 为需求冲击、δ_c 为成本冲击、δ_{comp} 为竞争价格冲击、$\bar{Q}$ 为可选库存上限，则：

$$\Delta \log p^{comp} = \log(1 + \delta_{comp}), \tag{26}$$

$$\widetilde{Q}(p', m') = \widehat{Q}_{anchor}\left(p', m'; \Delta \log p^{comp}\right)(1 + \delta_q), \tag{27}$$

$$Q^{sold}(p', m') = \min\{\widetilde{Q}(p', m'), \bar{Q}\}, \tag{28}$$

$$c^{eff} = c(1 + \delta_c), \tag{29}$$

$$Revenue(p', m') = p' Q^{sold}(p', m'), \tag{30}$$

$$Profit(p', m') = (p' - c^{eff})Q^{sold}(p', m') - Fm'. \tag{31}$$

这些 multiplicative shocks 必须满足：

$$\delta_q > -1, \qquad \delta_c > -1, \qquad \delta_{comp} > -1, \tag{32}$$

否则需求、成本或竞争价格会变成负数，且 $\Delta \log p^{comp}$ 不再有定义。app 的 risk flags 使用对称阈值：

$$|\delta_q| > 0.20, \quad |\delta_c| > 0.25, \quad |\delta_{comp}| > 0.15, \quad \text{or} \quad \bar{Q} < \bar{Q}^{base}_{cell}. \tag{33}$$

10.4 输出表

情景	有效价格	促销状态	预测销量	收入	利润
当前策略	p_0	m_0	Q_0	R_0	Π_0
用户输入策略	p'	m'	$\widehat{Q}$	$\widehat{R}$	$\widehat{\Pi}$
最优策略	p^*	m^*	$\widehat{Q}^*$	$\widehat{R}^*$	$\widehat{\Pi}^*$

11 优化器设计

11.1 目标函数

app 提供三个目标:

1. 最大化收入:

$$\max_{p,m} Revenue(p, m, p^{comp}). \tag{34}$$

2. 最大化利润:

$$\max_{p,m} Profit(p, m, p^{comp}). \tag{35}$$

3. 最大化利润且满足销量约束:

$$\max_{p,m} Profit(p, m, p^{comp}) \quad \text{s.t. } Q^{sold}(p, m, p^{comp}) \geq Q_{\min}. \tag{36}$$

11.2 业务约束

$$p_{\min} \leq p \leq p_{\max}, \qquad m \in \{0, 1\}, \qquad p - c \geq margin_{\min}, \tag{37}$$

$$Q^{sold}(p, m, p^{comp}) = \min\{\widehat{Q}_{anchor}(p, m, p^{comp}), Inventory\}. \tag{38}$$

约束必须存在，否则优化器会像玩具。真实业务中不能推荐负毛利、过低有效价格、超过库存承载能力或破坏品类价格带的策略。若预测需求超过库存，app 使用 Q^{sold} 计算可实现收入和利润，并把 stockout risk 作为 guardrail flag。若增强版加入连续折扣深度，再额外加入 $0 \leq d \leq d_{\max}$。

11.3 实现方式

MVP 用 grid search。因为 m 是二元变量，优化实际是两次一维搜索：先固定 $m = 0$ 扫描价格，再固定 $m = 1$ 扫描价格，最后比较两个最优利润。理由：透明、稳定、容易解释，面试中也能清楚说明复杂度。

增强版可用 `scipy.optimize`，但不是必要条件。先把业务闭环做完整，再提高算法复杂度。

上界绑定是诊断，不是部署建议

04 counterfactual 的核心警示是：98.5% 的 eligible cells 推荐价格贴在上界 guardrail。这不是“优化器很自信”的证据，而是 constant-elasticity demand 与当前 cost proxy 共同推出的诊断结果。对于 interior single-product optimum，若边际成本恒定且 $\varepsilon < -1$，constant-elasticity demand $Q = Ap^{\varepsilon}$ 推出 Lerner condition：

$$\frac{p-c}{p} = -\frac{1}{\varepsilon}.$$

当 $\varepsilon \approx -1.73$ 时，隐含无约束 margin 约为 58%，因此模型会机械地偏好高价。正确 framing 是 raise-and-test candidates，不是 production deployment recommendations。

12 不确定性与稳健性

12.1 必须展示不确定性

候选价格附近必须展示利润置信区间或敏感性区间。可选实现：

- 对需求模型参数做 bootstrap；
- 用弹性估计的置信区间模拟上下界；
- 对成本和库存做 scenario sensitivity；
- 比较不同模型设定下的候选价格是否稳定。

12.2 稳健性检查

检查项	目的
去除极端价格周	检查促销或清仓是否驱动弹性
比较 SKU FE、store FE、SKU-store FE	检查门店间价格差异是否污染自身价格弹性
加入 week/month fixed effects	控制季节性与全市场共同冲击
加入 promo 控制	避免把条件 sale-code effect 误记为价格效应；同时承认 promo 仍可能内生
加入竞争价格	检查 Cereals 品牌替代效应是否影响候选价格
交叉弹性符号检查	若 $\widehat{\beta}_{cross} \leq 0$，标记为不稳定估计；优化器使用敏感性区间或关闭交叉价格响应，而不是机械代入可疑系数
用 holdout 验证预测误差	防止模型只在样本内表现好
弹性上下界模拟	检查候选价格是否对参数敏感

额外建议加入 store-week fixed effects 规格，作为比 week FE 更严格的 confounding 检查：

$$\log(Q_{ist}) = \alpha_{is} + \lambda_{st} + \beta_{own}\log(P_{ist}^{eff}) + \theta Promo_{ist} + \epsilon_{ist}. \tag{39}$$

其中 λ_{st} 吸收本地需求冲击、门店层面的 campaign、特定周的库存或客流条件。这个规格很重要，因为 DFF 的 price / promotion 是 manager choice，且 `SALE` code 不是干净的 randomized treatment；manual 也提示，即使 sale code 没有设置，当周仍可能存在促销。store-week FE 会牺牲一部分 cross-price variation，但可以更严格检查 own-price elasticity 是否由 local store-week confounding 驱动。若竞争价格指数在该规格中被本地品类价格变动机械吸收、共线或不稳定，则 store-week-FE robustness 中应排除 P_{ist}^{comp}。

13 A/B test 验证模块

13.1 为什么必须有实验模块

观察性需求估计不能替代线上实验。候选价格上线前应通过小流量实验验证。实验模块让该 demo 从“模型练习”变成“可落地决策系统”。

13.2 实验输出

- treatment：推荐有效价格 / 促销状态；
- control：当前有效价格 / 促销状态；
- primary metric：profit per visitor 或 revenue per visitor；
- guardrail metrics：conversion、return rate、stockout、customer complaints；
- sample size；
- test duration；
- randomization unit：user、session、geo、SKU-cluster。

13.3 样本量公式

$$n = \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2\sigma^2}{\Delta^2}. \tag{40}$$

其中 α 是显著性水平，$1-\beta$ 是 power，Δ 是最小可检测效应（MDE），σ^2 是主要指标方差。

14 Web app 设计

14.1 技术选择

技术栈

使用 Python + Streamlit。不要一开始做 React 全栈。求职信号来自模型闭环和业务解释，不来自前端复杂度。

层级	工具
数据处理	pandas, numpy
统计建模	statsmodels, linearmodels（可选）
机器学习 benchmark	scikit-learn（可选）
优化	scipy.optimize 或 grid search
可视化	plotly 或 matplotlib
Web app	Streamlit
部署	Streamlit Community Cloud / Render
版本管理	GitHub
文档	README + methodology doc + model card + executive memo

14.2 页面结构

页面	用户操作	输出
Demand Model	查看模型结果	自身弹性、交叉弹性、sale-code log-point effect、holdout error、识别 caveat
Counterfactual Simulator	调整 effective price、promo、competitor price、cost、inventory	capped sold units、收入、利润、变化率、风险提示
Profit Optimizer	点击 rerun optimizer 或调整 scenario	p^*、m^*、$\Delta Revenue$、$\Delta Profit$、upper-guardrail flag
Experiment Design	查看候选实验或输入 MDE / power	样本量、实验周期、guardrail metrics、underpowered flag
Limitations	阅读模型限制	价格/促销内生性、AAC cost proxy、库存场景化、raise-and-test framing
Upload & Score	上传业务方 CSV	字段校验、scenario scoring、候选动作和风险提示

14.3 关键交互

- slider：new effective unit price；
- slider：competitor price per oz；
- toggle：promo on / off；
- number input：unit cost；
- number input：promo fixed cost；
- number input：inventory；

- selectbox：objective（revenue / profit / constrained profit）；
- button：Run Counterfactual；
- button：Find Optimal Price / Re-run optimizer；
- file uploader：上传自有 SKU 数据并映射到 app schema；
- warning box：observational estimates require experimental validation。

15 关键图表

图表	作用
Historical price and quantity	展示 panel 数据结构和时间变化
Price vs quantity scatter	直观展示相关性，但注明不是因果
Estimated demand curve	展示价格到需求的函数关系
Revenue curve	显示收入最大化价格
Profit curve	显示利润最大化价格；这是核心图
Baseline vs raise-and-test candidate	把模型结果翻译成业务指标，同时标注部署风险
Uncertainty band around profit curve	展示模型风险和决策稳健性
Sample size curve	展示 A/B test 所需样本随 MDE 变化

16 项目文件结构

```
pricing-decision-engine/
|
|-- app.py
|-- README.md
|-- requirements.txt
|-- pricing_promotion_decision_engine_plan.tex
|-- pricing_promotion_decision_engine_plan.pdf
|
|-- docs/
| |-- app_wireframe.md
| |-- methodology.md
| |-- methodology.tex
| |-- methodology.pdf
|
|-- rawData/
| |-- README.md
```

```
| |-- dominicks_manual.pdf
| |-- wcer.csv
| |-- upccer.csv
|
|-- data/
| |-- processed/
| |-- sku_store_week_panel.parquet
| |-- brand_size_store_week_panel.parquet
| |-- demand_modeling_dataset.parquet
| |-- model_coefficients.csv
| |-- top_recommendations.csv
| |-- experiment_candidates.csv
|
|-- notebooks/
| |-- 01_data_cleaning.ipynb
| |-- 02_eda.ipynb
| |-- 03_demand_estimation.ipynb
| |-- 04_counterfactual.ipynb
| |-- 05_ab_testing_design.ipynb
|
|-- pages/
| |-- 1_Demand_Model.py
| |-- 2_Counterfactual_Simulator.py
| |-- 3_Profit_Optimizer.py
| |-- 4_Experiment_Design.py
| |-- 5_Limitations.py
| |-- 6_Upload_and_Score.py
|
|-- src/
| |-- data.py
| |-- features.py
| |-- validation.py
| |-- scenario.py
| |-- simulation.py
| |-- optimization.py
| |-- plots.py
| |-- upload.py
|
|-- tests/
| |-- test_features.py
| |-- test_scenario.py
| |-- test_optimization.py
| |-- test_upload.py
|
|-- reports/
   |-- data_cleaning_summary.md
```

```
|-- eda_summary.md
|-- demand_model_summary.md
|-- counterfactual_summary.md
|-- ab_test_plan.md
|-- figures/
```

17 README 结构

README 不应写成技术流水账，而应写成商业 memo + 技术说明。

根目录 `README.md` 是整个项目的首页；`rawData/README.md` 只写原始数据来源、文件清单、字段字典、manual 摘要和读取规则；`reports/data_cleaning_summary.md` 记录真实清洗诊断与样本规模。不要把根 README 写成 codebook，也不要把 raw data README 写成项目 pitch。

17.1 必须包含的部分

1. **Business Problem**：定价和促销如何影响需求、收入、利润和库存风险。
2. **Data**：数据来源、粒度、商品数量、观察值数量、字段说明、真实字段与 scenario inputs。
3. **Relation to Prior Work**：说明项目与 DFF scanner-data 研究、cereal demand 文献、revenue optimization 和 experimentation 的关系；强调这是 applied decision product，不是新结构估计器。
4. **Method**：需求估计、反事实模拟、约束优化、A/B test 设计。
5. **Results**：弹性、raise-and-test 候选价格、收入变化、利润变化、样本量。
6. **How to Run**：安装依赖和启动 app。
7. **Limitations**：价格和促销内生性、竞争价格构造假设、库存场景化、上线前需要实验验证。

17.2 README 必须写明的模型限制

> The aggregate cross-price term approximates average substitution intensity across competing cereal brands. A full brand-pair substitution matrix would require a discrete-choice or random-coefficients structural demand model and is outside the MVP scope.
>
> This app is a transparent decision-support implementation inspired by scanner-data demand estimation and pricing optimization. It should not be presented as a new econometric estimator or as a replication of Nevo's structural cereal model.
>
> The competitor price index uses baseline or lagged sales-share weights rather than current-week sales shares, reducing mechanical endogeneity from contemporaneous demand shocks. Results should still be interpreted as decision support, not final causal evidence.

17.3 README 结果表模板

Result	Value
Estimated own-price elasticity	来自模型，不手填
Estimated cross-price elasticity	来自模型，不手填
Promo sale-code effect	来自模型，不手填；若以百分比展示，使用 $\exp(\theta) - 1$
Raise-and-test candidate price	来自优化器
Expected revenue change	来自反事实模拟
Expected profit change	来自反事实模拟
Required sample size for A/B test	来自实验模块

18 执行路线图

阶段	任务	具体动作	产出
1	数据选择与清洗	已完成 DFF Cereals 清洗、SALE code 处理、unit price / cost proxy 构造	SKU-store panel
2	EDA 与聚合	已完成 brand mapping、size parsing、brand-size panel、粒度选择诊断	`02_eda.ipynb` + EDA reports
3	需求估计	已估计 log demand with SKU-store FE, week FE, own price, competitor price, promo	coefficients + holdout metrics
4	反事实函数	已实现 anchored counterfactual，默认固定 competitor price，可接受 scenario shock	`simulation.py` + `scenario.py`
5	优化器	已实现 constrained grid search，含 price guardrail、margin、inventory、promo cost	`optimization.py`
6	实验模块	已生成 A/B validation design、sample size curve、risk flags	05 notebook
7	Streamlit app	已搭建六页 MVP，含 simulator、optimizer、limitations、upload scoring	local Streamlit app
8	文档同步	已同步 README、methodology、reports 和本计划的公式口径	GitHub-ready docs
9	下一步 A	检查同品牌跨包装规格 cannibalization，判断 98.5% guardrail binding 是否稳健	07 notebook
10	下一步 B/C	补 tests、部署、case study；可选 Hausman IV / cost-shock IV	portfolio link + causal-IV extension

19 一页 Executive Memo 模板

Problem. Pricing and promotion decisions affect demand, revenue, margin, and inventory risk. The current analysis builds a decision engine to evaluate pricing counterfactuals and generate constrained raise-and-test pricing candidates.

Relation to prior work. The project is positioned as an applied decision product built on scanner-data demand estimation, cereal differentiation research, revenue optimization, and online experimentation, not as a new econometric estimator.

Method. Estimate demand response using SKU-store-week panel data with SKU-store and week fixed effects, own price, competitor price, and promotion controls; use Duan smearing only for log-demand level prediction; use cell-anchored relative response for counterfactual optimization; compute revenue and profit from inventory-capped sold units; design an A/B test for validation.

Candidate action. Report the raise-and-test effective price and promotion status, expected revenue/profit change, upper-guardrail flag, and decision risk. Values must be generated by the app.

Caveat. Observational price and promotion variation is not final causal evidence. Candidate actions should be validated with randomized or quasi-experimental rollout.

20 简历 bullet 与面试话术

20.1 简历 bullet

Built an interactive pricing and promotion decision engine using Python, Streamlit, and SKU-store-week panel demand estimation; estimated own- and cross-price elasticity, simulated effective price and promotion counterfactuals, generated constrained raise-and-test pricing candidates under cost and inventory constraints, and designed an A/B testing framework for validation.

如果已有真实计算结果，可以改为：

Built an interactive pricing and promotion decision engine using Python, Streamlit, and SKU-store-week panel data; estimated own- and cross-price elasticity, flagged upper-guardrail optimizer behavior, and quantified expected revenue and margin changes under alternative pricing and promotion scenarios.

不要写无法证明的百分比。百分比必须来自 app 的实际输出。

20.2 60 秒面试版本

I built a pricing decision engine for retail products using Dominick's SKU-store-week scanner data. It is positioned as an applied decision product, not a new econometric estimator: the app takes ideas from scanner-data demand estimation, cereal differentiation research, and revenue

optimization, then estimates own-price elasticity, cross-price elasticity, and promotion response with fixed effects. It simulates counterfactual pricing and promotion strategies, generates constrained raise-and-test candidates under cost and inventory constraints, and includes an A/B testing module because observational price and promotion variation is not enough for causal rollout decisions.

21 关键风险清单

1. 变成销量预测项目。预测只是输入，不是最终产品。
2. 没有利润函数。只看 revenue 会被定价团队质疑。
3. 忽略价格和促销内生性。必须写 caveat、识别诊断和实验验证方案。
4. 没有约束。真实定价有库存、成本、价格下限、最低 margin。
5. 没有 **web app**。notebook 不足以证明决策工具化能力。
6. 没有 **README** 和 **memo**。招聘方不会自己读代码推断你的商业价值。

22 最终交付物清单

交付物	内容	优先级
Streamlit web app	交互式定价、促销、反事实、优化、实验设计	必须
GitHub repo	可运行代码、清楚结构、requirements、核心函数 tests	必须
README	商业问题、数据、方法、结果、限制、运行方式	必须
Methodology doc	需求模型、反事实模拟、利润函数、优化器、识别 caveat，提供 Markdown 与 TeX 两版	必须
Executive memo	一页业务结论，适合发给内推或 recruiter	必须
Limitations / model card 页面	数据限制、识别限制、适用范围、guardrail binding 风险	建议
Demo video/GIF	60-90 秒演示 app 如何使用	建议
Resume bullet	对齐定价、因果、优化、实验	必须

23 最终判断

这个 demo 的正确形式是：

核心闭环

数据 → 需求估计 → 反事实模拟 → 利润优化 → 实验验证 → 业务建议

最终输出必须让招聘方在 60 秒内看懂：你不是只会做学术结构模型，也不是只会做 dashboard；你能把经济学、因果推断和数据科学压缩成一个直接影响价格、促销和利润的决策工具。

参考文献

1. Angrist, Joshua D., and Jörn-Steffen Pischke. 2009. *Mostly Harmless Econometrics: An Empiricist's Companion*. Princeton University Press.
2. Berry, Steven, James Levinsohn, and Ariel Pakes. 1995. "Automobile Prices in Market Equilibrium." *Econometrica* 63(4): 841–890. https://doi.org/10.2307/2171802.
3. Duan, Naihua. 1983. "Smearing Estimate: A Nonparametric Retransformation Method." *Journal of the American Statistical Association* 78(383): 605–610. https://doi.org/10.1080/01621459.1983.10478017.
4. Hendel, Igal, and Aviv Nevo. 2006. "Sales and Consumer Inventory." *RAND Journal of Economics* 37(3): 543–561. https://doi.org/10.1111/j.1756-2171.2006.tb00030.x.
5. Imbens, Guido W., and Donald B. Rubin. 2015. *Causal Inference for Statistics, Social, and Biomedical Sciences*. Cambridge University Press.
6. Kohavi, Ron, Diane Tang, and Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.
7. Nevo, Aviv. 2001. "Measuring Market Power in the Ready-to-Eat Cereal Industry." *Econometrica* 69(2): 307–342. https://doi.org/10.1111/1468-0262.00194.
8. Phillips, Robert L. 2005. *Pricing and Revenue Optimization*. Stanford University Press.
9. Talluri, Kalyan T., and Garrett J. van Ryzin. 2004. *The Theory and Practice of Revenue Management*. Springer.
10. Train, Kenneth E. 2009. *Discrete Choice Methods with Simulation*. Cambridge University Press.
11. UChicago Booth Kilts Center for Marketing. n.d. "Dominick's Finer Foods Database." Accessed April 2026. https://www.chicagobooth.edu/research/kilts/research-data/dominicks.